Adaptive Multi-Layer Guardrail for Prompt Injection in LLM Applications: Project Report

Abstract

This project builds and tests a layered guardrail that stops prompt-injection attacks before they reach a large language model (LLM). It combines a long-context ModernBERT classifier, a multilingual novelty detector and a perplexity detector into one calibrated risk score, retrains the system against a black-box mutation attacker, and is evaluated over three random seeds on long documents, unseen document formats, held-out attack types and other public datasets. Its clearest results are detection of injections hidden in long documents and resistance to adaptive attacks; its limits are equally reported: accuracy no better than a retrained DistilBERT, document false alarms, and poor transfer to other datasets.

Item	Summary
Problem	LLM applications cannot reliably separate trusted instructions from untrusted text. Attackers hide instructions in prompts, emails, web pages and documents, in encoded or foreign-language form, and long documents hide them where short-input detectors never look.
Solution	A three-layer detector (supervised, novelty, statistical) fused into a calibrated score with an allow, review or block decision, and hardened by retraining against an automatic adaptive attacker.
AI used	Transformer fine-tuning with a multi-task head, sentence embeddings, language-model perplexity, gradient boosting, probability calibration, automated adversarial attack generation, SHAP explanations.

Item	Summary
AI models	ModernBERT-base (Layer 1), multilingual-e5-small (Layer 2), Qwen2.5-0.5B (Layer 3), XGBoost meta-classifier. Baselines: TF-IDF with SVM and logistic regression, DistilBERT, DeBERTa-v3-small, and a pretrained guard model (protectai/deberta-v3-base-prompt-injection-v2).
Dataset	deepset/prompt-injections (real prompts), plus generated harmless documents in sixteen formats, about 100 look-alike harmless prompts, attacks hidden in documents and mutation-generated attacks. Other datasets for transfer: xTRam1/safe-guard-prompt-injection and jackhhao/jailbreak-classification.
Accuracy (3 seeds, mean ± std)	Clean test: F1 0.940 ± 0.032, ROC-AUC 0.984 ± 0.003. Adaptive attack success: 1.6% ± 1.6% for the fused system, against 44% ± 31% for DistilBERT and 96% ± 5% for TF-IDF with SVM. Injections hidden in 2,000 to 12,000 character documents: 100% detected with at most 1% false alarms. Limits: 76% ± 9% of clean-test attacks blocked at the operating threshold, 12% ± 16% of harmless documents flagged, ROC-AUC 0.48 on xTRam1.

Keywords: prompt injection, jailbreak detection, LLM security, ModernBERT, adversarial training, calibration.

1. Introduction

Prompt injection is the top practical weakness of LLM applications, because the model treats instructions and data as one stream of text. A support bot, an email assistant or a retrieval-augmented generation (RAG) agent reads untrusted content, and any instruction hidden in that content can steer the model.

1.1 Attack types

- Direct prompt injection: the user types text such as "ignore previous instructions" to override the system prompt or leak it.

- Indirect prompt injection: the instruction sits inside data the application fetches, such as a web page, a PDF or an email, and the victim never sees it.
- Jailbreaks: role-play, hypothetical framing or adversarial suffixes that push an aligned model past its safety training.
- Obfuscated attacks: the same intent hidden by base64, leetspeak, translation, look-alike characters or unusual formatting.

1.2 Use case

The detector sits in front of an LLM application as an input guard. Every user message and every retrieved document is scored before it reaches the model. Low-risk input is allowed, high-risk input is blocked, and uncertain input can be sent to review or handled with restricted tool access. Typical deployments are customer-support chatbots, email and calendar assistants, document-summarisation services and tool-using agents.

1.3 Why simple defenses fail

Keyword and regular-expression filters miss paraphrases and other languages. A single classifier trained on one dataset learns that dataset's style and drops sharply on unseen attack families. Attackers can probe a deployed filter and adapt, so a defense measured only on a fixed test set overstates its strength. Long documents add a further problem: most classifiers read only the first few hundred tokens, so an instruction placed deep inside a page is never seen.

1.4 Objectives

- Detect direct and indirect prompt injection with high recall and a low false-positive rate.
- Make detection robust by fusing complementary signals instead of relying on one model.
- Handle long documents and several languages.
- Test the system against an adaptive attacker and improve it through adversarial retraining.
- Compare it with classical, transformer and pretrained-guard baselines using the same metrics, and report failures honestly.
- Explain individual detections so a security analyst can audit them.

1.5 Contributions

1. A three-layer detector with a calibrated score and a three-way allow, review or block policy.
2. A long-document protocol and the finding that chunked reading is needed even for an 8,192-token model.
3. A black-box mutation attacker with a validity check and held-out attack operators, used for both retraining and evaluation.
4. A three-seed evaluation including layer ablation, transfer to other datasets, speed, and the outcome of every hypothesis stated in advance.
5. A preliminary first version (v1) whose measured failures, such as a collapsed classifier and over-blocking of harmless documents, directly shaped the final design (Section 4.7).

2. Literature Review

The cited work shows that prompt injection is easy to launch, hard to filter, and measurable with public benchmarks. Numbers in brackets refer to the reference list at the end.

2.1 Attacks and threat models

- [1] Perez and Ribeiro, "Ignore Previous Prompt" (2022). Introduces the PromptInject framework and two attack goals: goal hijacking (changing the task) and prompt leaking (exposing the system prompt). Tested on GPT-3, it shows that even low-skill attackers succeed because model output is stochastic.
- [2] Greshake et al., "Not what you've signed up for" (2023). Defines indirect prompt injection, where instructions are planted in data the application retrieves. It lists threats such as data theft, worming, information-ecosystem contamination and remote control, and demonstrates them on real LLM-integrated applications.
- [3] Liu et al., "Prompt Injection Attacks and Defenses in LLM-Integrated Applications" (2023). Gives a formal framework for prompt injection and shows that a combined attack outperforms single techniques. It also benchmarks several defenses across multiple LLMs and tasks.
- [4] Schulhoff et al., HackAPrompt (EMNLP 2023). Reports a global prompt-hacking competition that collected more than 600,000 adversarial prompts against three LLMs. It contributes a public dataset and a taxonomy of prompt-hacking techniques.

- [5] Zou et al., "Universal and Transferable Adversarial Attacks" (2023). Uses greedy, gradient-guided search to find adversarial suffixes that make aligned models produce harmful output. The suffixes transfer to other models, including closed ones.
- [6] Chao et al., PAIR (2023). Uses an attacker LLM that refines a jailbreak prompt step by step against a black-box target, often within about twenty queries. This is the template for the score-guided attacker in this project.

2.2 Benchmarks and detection

- [7] Yi et al., BIPIA (2023). The first benchmark for indirect injection, covering email, web and table question answering and summarisation. It finds that models cannot reliably tell instructions from external content, and proposes prompt-based and fine-tuning defenses.
- [8] Debenedetti et al., AgentDojo (NeurIPS 2024 Datasets and Benchmarks). A dynamic environment with 97 realistic tasks and 629 security test cases for tool-using agents. It shows that agents often fail tasks even without attacks, and that existing attacks and defenses leave room for improvement.
- [9] Alon and Kamfonas, "Detecting Language Model Attacks with Perplexity" (2023). Observes that adversarial suffixes have unusually high perplexity. A lightweight classifier on perplexity and token length separates them from normal prompts, which motivates Layer 3 here.

2.3 Models, calibration and explainability

- [10] Warner et al., ModernBERT (2024). A modernised BERT-style encoder trained on about two trillion tokens with a native 8,192-token context, rotary position embeddings and alternating local and global attention. It is the main classifier here.
- [11] He et al., DeBERTaV3 (ICLR 2023). Improves DeBERTa with ELECTRA-style replaced-token-detection pretraining and gradient-disentangled embedding sharing. It is used as a comparison model.
- [12] Sanh et al., DistilBERT (2019). A distilled BERT that is about 40% smaller and 60% faster while keeping most of BERT's language understanding. It is the lightweight transformer baseline.
- [13] Wang et al., Multilingual E5 text embeddings (2024). Sentence embeddings for more than 90 languages that can be compared by cosine similarity. They supply the vectors for the novelty detector (Reimers and Gurevych [14] introduced sentence embeddings of this kind).

- [14] Reimers and Gurevych, Sentence-BERT (EMNLP 2019). Siamese-network sentence embeddings compared by cosine similarity, the idea behind the novelty layer.
- [15] Qwen Team, Qwen2.5 technical report (2024). The small multilingual language model used to compute perplexity in Layer 3.
- [16] Chen and Guestrin, XGBoost (KDD 2016). Scalable gradient-boosted trees, used as the fusion model.
- [17] Platt, probabilistic outputs for support vector machines (1999). Fitting a logistic function to scores turns them into calibrated probabilities, as done here.
- [18] Lundberg and Lee, SHAP (NeurIPS 2017). Assigns each input feature a contribution to a prediction using Shapley values. It is used to explain which tokens trigger a detection.

2.4 Research gap

Most prior detectors are single models evaluated on fixed test sets, and most attack papers do not test how a defense recovers after retraining. Few studies check detection of injections buried deep in long documents, and few report whether a detector transfers to other public datasets. This project addresses those gaps by fusing complementary detectors, testing against a score-guided adaptive attacker, measuring long-document behaviour, and reporting transfer failures openly.

3. System Design and Features

The system scores every input with three independent detectors, fuses their outputs into one calibrated probability and applies a decision policy. Long inputs are cut into overlapping chunks of 900 characters (200 overlap) and the highest chunk score decides.

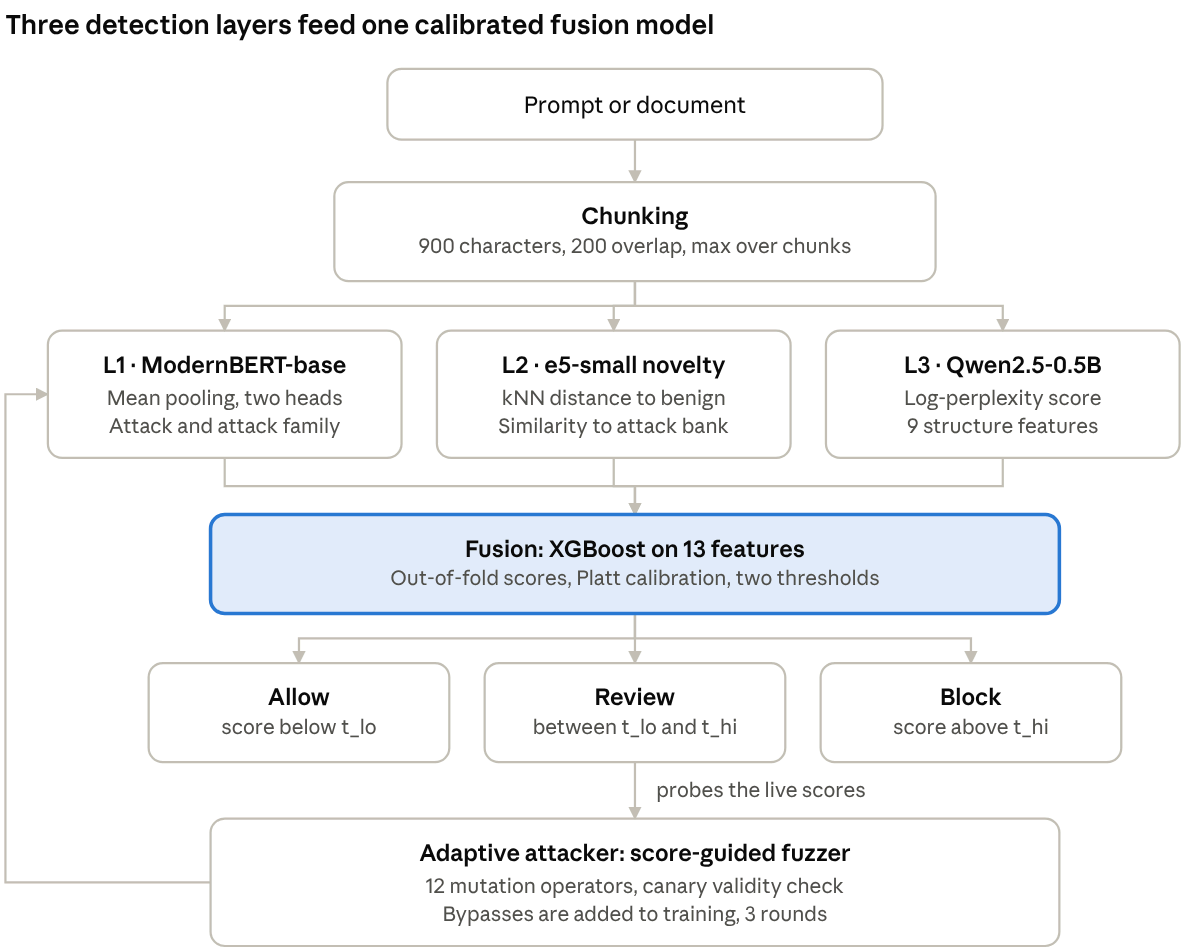


Figure 0. System architecture · 3 layers, fusion, allow/review/block, attacker loop

How it works: each input is cut into overlapping chunks, every chunk is scored by the three layers, and the highest chunk score is kept. Fusion turns the 13 layer outputs into one calibrated probability, and two thresholds map it to allow, review or block. In evaluation, the adaptive attacker probes the scores and its bypasses are added to training in three retraining rounds.

3.1 The three layers

- Layer 1, supervised classifier: ModernBERT-base with mean pooling and two output heads. The main head decides attack or benign. A second head predicts the attack family (override, role-play, base64, leetspeak, multilingual, hidden-in-document, suffix, combined) with an auxiliary loss weight of 0.3, which forces the encoder to learn attack structure and not only surface words.

- Layer 2, novelty detector: multilingual sentence embeddings (multilingual-e5-small). Two features: the distance to the nearest harmless training traffic (average of the five nearest neighbours) and the highest similarity to a bank of known attacks. It reacts to inputs unlike anything seen as harmless.
- Layer 3, statistical detector: log-perplexity from a small multilingual language model (Qwen2.5-0.5B), plus nine structure features: length, share of non-ASCII characters, punctuation, digits, capitals, base64-like spans, number of scripts, zero-width characters and average word length. It targets gibberish suffixes and obfuscation.

3.2 Fusion, calibration and decision policy

1. The 13 features (1 from Layer 1, 2 from Layer 2, 10 from Layer 3) feed a gradient-boosted meta-classifier (XGBoost).
2. The raw score is calibrated with Platt scaling fitted on out-of-fold validation scores, so 0.9 means about nine attacks in ten.
3. Two thresholds define the policy. Below t_lo (the score that keeps 95% of attacks) the input is allowed. Above t_hi (the score that keeps harmless false positives at or below 1%) it is blocked. Between them it goes to review or restricted handling. In the final runs the two thresholds coincided, so review stayed empty (Section 5.7).

3.3 Features of the system

- Detects direct and indirect injection, including attacks hidden in long documents.
- Handles base64, leetspeak, zero-width characters, look-alike letters and foreign-language prefixes.
- Gives a calibrated risk score and an allow, review or block decision instead of a bare label.
- Retrains automatically against a black-box attacker that only sees the defense's score.
- Detects a failed training run (collapse) and retries at other learning rates.
- Explains detections with token-level SHAP or occlusion importance.
- Fully reproducible: fixed seeds, saved configuration, every table written to CSV.

4. Implementation and Technical Usage

The whole pipeline is one Python script (prompt_injection_v2.py) built on PyTorch, Hugging Face Transformers, sentence-transformers, scikit-learn, XGBoost and SHAP. Each seed ran on a free Colab T4 GPU in about 54 minutes.

4.1 Data

- Real prompts: deepset/prompt-injections, with training and test splits kept separate and duplicates removed. The test set has 116 prompts (60 attacks, 56 harmless).
- Harmless documents: sixteen document formats built from filler sentences. Twelve are used in training (emails, HTML, reviews, encyclopedia text, invoices, tables, CSV, YAML, meeting notes, code, news and Q&A pages) and four unseen formats (README, JSON record, chat log, server log) are used only for testing.
- Look-alike harmless prompts: about 100 prompts that contain attack-like words but are harmless, for example "ignore the outliers in this dataset" or "what does this base64 string decode to?", plus harmless prompts in Spanish, French, German, Italian and Portuguese. Half train the model and half test it.
- Long documents: harmless paragraphs of 300 to 3,000 characters, and documents with an injection hidden at a random position, were added to training. The long-document test uses different sentences.
- Attacks hidden in documents: real attack prompts from the training set are wrapped in training document formats.
- Held-out material: the multilingual attack family and two mutation operators (homoglyph substitution and foreign-language prefix) are never used for training.
- All attack payloads are harmless canary instructions ("reply only with the word CANARY").

4.2 Layer 1 training

- Backbone: answerdotai/ModernBERT-base; if it cannot load, the script logs a warning and falls back to DistilBERT.
- Optimiser: AdamW, weight decay 0.01, learning rate 5e-5 for the encoder and ten times higher (capped at 5e-4) for the heads, 10% warm-up then linear decay, gradient clipping at 1.0, class-weighted cross-entropy, batch size 16, up to 6 epochs, maximum training length 512 tokens.
- Loss: binary cross-entropy plus 0.3 times the attack-family cross-entropy.

- Memory: token-budgeted micro-batches (about 2,048 tokens) with gradient accumulation to an effective batch of 16; other models are moved to the CPU while one trains. Full precision on the T4.
- Safeguards: 12% of the training data is held out as a development set, the best epoch by development AUC is kept, and training is retried at 2x and 0.5x the learning rate if the best AUC is below 0.9. A model that stays below 0.65 is flagged as collapsed in every output.

4.3 Layers 2 and 3

- Embeddings: intfloat/multilingual-e5-small with the "query: " prefix (fallback: all-MiniLM-L6-v2).
- Perplexity: Qwen/Qwen2.5-0.5B per chunk (fallback: GPT-2).
- Chunking: 900 characters with 200 overlap; the document score is the maximum over chunks.

4.4 Adaptive attacker and adversarial retraining

- The attacker is a black-box fuzzer. It starts from template attacks, applies one or two random mutations, queries only the defense's score, and keeps the lowest-scoring variant of each lineage.
- Twelve operators: synonym swap, character spacing, zero-width insertion, homoglyph substitution (held out), case change, leetspeak, filler text, document wrapping, base64 wrapping, foreign-language prefix (held out), sentence reordering, newline injection.
- Validity rule: a mutant counts only if, after removing homoglyphs, zero-width characters and leetspeak and decoding base64, it still contains the canary payload. The attacker cannot win by destroying the attack.
- Each of the three retraining rounds adds the successful bypasses and the hardest near-misses to the training data, together with the same number of fresh harmless examples. The balanced harmless examples prevent the over-blocking seen in the first version.

4.5 Evaluation protocol

- Clean metrics: accuracy, precision, recall, F1, ROC-AUC, PR-AUC, true-positive rate at 1% and 5% false-positive rate, expected calibration error and Brier score.

- Attack success rate (ASR): the share of attacks scored below the operating threshold. Reported for template families, the held-out family, indirect attacks in unseen document formats, combined attacks and the adaptive fuzzer. Lower is better.
- False positive rate (FPR): measured separately on clean prompts, look-alike prompts and harmless documents.
- Long documents: injections at 10%, 50% and 90% of documents of 2,000, 6,000 and 12,000 characters, read three ways (first 512 tokens, whole document, chunks).
- Layer ablation, transfer to two other public datasets, latency by input length, and three seeds (42, 43, 44) with mean and standard deviation.
- Fair comparison: the TF-IDF and DistilBERT baselines are also retrained on the final adversarial data.

4.6 Technical usage

1. Choose a GPU runtime with about 15 GB of memory (free Colab T4) and install: pip install -U transformers datasets sentence-transformers scikit-learn xgboost shap matplotlib pandas sentencepiece accelerate. If DeBERTa's tokenizer fails with a protobuf message, run pip install -U protobuf and restart the runtime.
2. Logic test without downloads (about 20 seconds): python prompt_injection_v2.py --smoke --out smoke_test
3. One full seed: python prompt_injection_v2.py --seed 42 --cross --guard-model protectai/deberta-v3-base-prompt-injection-v2 --out results_s42 (repeat with seeds 43 and 44).
4. Check layer1_diagnostics.csv first: the development AUC of ModernBERT must be well above 0.9.
5. Outputs per seed: model_comparison.csv, adaptive_attack.csv, robustness_rounds.csv, long_doc.csv, policy.csv, ablation.csv, cross_dataset.csv, latency.csv, error_analysis.csv, run_config.json and eight figures.
6. For deployment, call the fused scorer on each input and act on the policy: allow, review or block.

An NVIDIA GPU with roughly 8 GB or more is needed; a 4 GB laptop GPU is too small for the training step.

4.7 Preliminary study (version 1) and lessons

The first version used DeBERTa-v3-small, an English-only perplexity model, and a fixed-template attacker. Over three seeds its DeBERTa classifier predicted one class for every input (ROC-AUC 0.500), so every fused result reflected only the other two layers. The fused system blocked about 94% of harmless documents and 56% of look-alike prompts, so its low attack success rate was largely over-blocking, and the template attacker found bypasses only in the first round. Each of these findings drove one change in version 2: collapse detection with retries, harmless documents and look-alikes in training, multilingual embeddings and perplexity, and the mutation attacker.

5. Results

The fused system detects every injection hidden in long documents and is the hardest model to bypass adaptively, but it is not more accurate than a retrained DistilBERT and it does not transfer to other datasets. All values are mean ± standard deviation over seeds 42, 43 and 44. Figures come from seed 42. No model collapsed: ModernBERT's development AUC was 0.997 to 1.000 in every round and seed.

5.1 Clean test set

Model	F1	ROC-AUC	TPR at 1% FPR	TPR at 5% FPR	Recall at val. threshold	FPR at val. threshold
TF-IDF + LinearSVM	0.834 ± 0.004	0.943 ± 0.009	0.683 ± 0.073	0.828 ± 0.025	0.817 ± 0.076	0.071 ± 0.078
DistilBERT	0.935 ± 0.019	0.989 ± 0.007	0.950 ± 0.000	0.961 ± 0.010	0.911 ± 0.010	0.000 ± 0.000
DeBERTa-v3-small	0.945 ± 0.015	0.990 ± 0.011	0.917 ± 0.033	0.939 ± 0.042	0.878 ± 0.035	0.006 ± 0.010
Pretrained guard model	0.537	0.896	0.533	0.717	0.867 ± 0.029	0.357 ± 0.093
ModernBERT alone (initial)	0.941 ± 0.014	0.988 ± 0.006	0.883 ± 0.076	0.967 ± 0.017	0.922 ± 0.025	0.006 ± 0.010
Fused system, round 0	0.944 ± 0.015	0.989 ± 0.005	0.878 ± 0.075	0.961 ± 0.038	0.928 ± 0.010	0.012 ± 0.010
Fused system, final	0.940 ± 0.032	0.984 ± 0.003	0.856 ± 0.051	0.928 ± 0.054	0.761 ± 0.092	0.000 ± 0.000

Model	F1	ROC-AUC	TPR at 1% FPR	TPR at 5% FPR	Recall at val. threshold	FPR at val. threshold
DistilBERT + adversarial data	0.932 ± 0.011	0.979 ± 0.002	0.911 ± 0.025	0.928 ± 0.010	0.828 ± 0.059	0.000 ± 0.000

DistilBERT, DeBERTa, ModernBERT and the fused system are within about one standard deviation of each other on a 116-prompt test set, so none is shown to be more accurate; all beat TF-IDF clearly. The pretrained guard model flags 36% of harmless deepset prompts. The final fused system reaches only 76% recall at its threshold, not the 95% target: the threshold is set on a validation mix with many easy generated attacks, so it is too high for the harder real attacks in the test set.

5.2 Adaptive attack

The attacker sees only the defense's score. A success counts only if the mutant still contains the canary payload after normalisation.

System	Static attack success	Adaptive attack success	Held-out multilingual family
Fused system, final	0.010 ± 0.018	0.016 ± 0.016	0.083 ± 0.144
ModernBERT alone, final	0.016 ± 0.027	0.047 ± 0.068	0.250 ± 0.331
Pretrained guard model	0.000	0.000	0.000
DistilBERT	0.088 ± 0.104	0.438 ± 0.312	0.458 ± 0.382
DeBERTa-v3-small	0.094 ± 0.087	0.495 ± 0.423	0.708 ± 0.289
DistilBERT + adversarial data	0.021 ± 0.018	0.115 ± 0.126	0.292 ± 0.260
TF-IDF + LinearSVM	0.135 ± 0.092	0.958 ± 0.048	1.000
TF-IDF + LinearSVM + adversarial data	0.021 ± 0.036	0.219 ± 0.081	0.917 ± 0.072

DistilBERT and DeBERTa trained on the same initial data were bypassed in 12.5% to 94% of cases depending on the seed, while the fused system stayed between 0% and 3.1% in all three seeds and ModernBERT alone between 0% and 12.5%. Adversarial data lowers the attack success of DistilBERT and TF-IDF. The pretrained guard model was never bypassed but over-blocks harmless prompts. The multilingual family, never used for

training, is the weakest point. The two operators never used for training (homoglyphs and foreign-language prefix) succeeded against the fused system only 1.4% and 0% of the time. With 64 attacks per system, differences of a few points are not significant.

5.3 Adversarial retraining rounds (fused system)

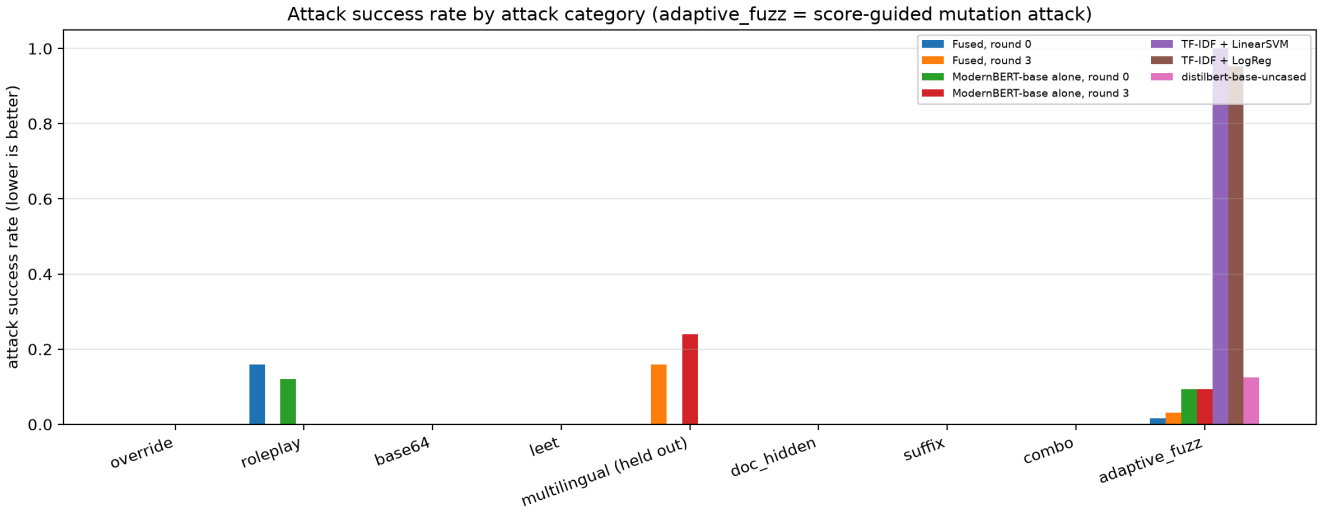


Figure 1. Attack success rate by attack category for each detector (seed 42).

Round	Harmless documents flagged	Unseen-format documents flagged	Look-alike prompts flagged	Template attack success	Held-out multilingual success	Hidden-in-document success	Adaptive attack success
0	0.356 ± 0.190	0.289 ± 0.126	0.073 ± 0.036	0.024 ± 0.014	0.000	0.000	0.052 ± 0.039
1	0.389 ± 0.077	0.394 ± 0.059	0.083 ± 0.036	0.011 ± 0.009	0.000	0.000	0.026 ± 0.024
2	0.261 ± 0.118	0.211 ± 0.084	0.021 ± 0.018	0.000	0.040 ± 0.040	0.033 ± 0.044	0.167 ± 0.275
3	0.117 ± 0.161	0.144 ± 0.151	0.010 ± 0.018	0.000	0.053 ± 0.092	0.067 ± 0.060	0.031 ± 0.031

Retraining with balanced harmless examples cut false alarms on harmless documents from 36% to 12% and on look-alike prompts from 7% to 1%. The price is a small rise in held-out multilingual and hidden-in-document attack success (0% to 5% and 7%). The attacker found only 0 to 7 bypasses among 210 seeds per round, so the retraining signal is weak.

5.4 Long documents

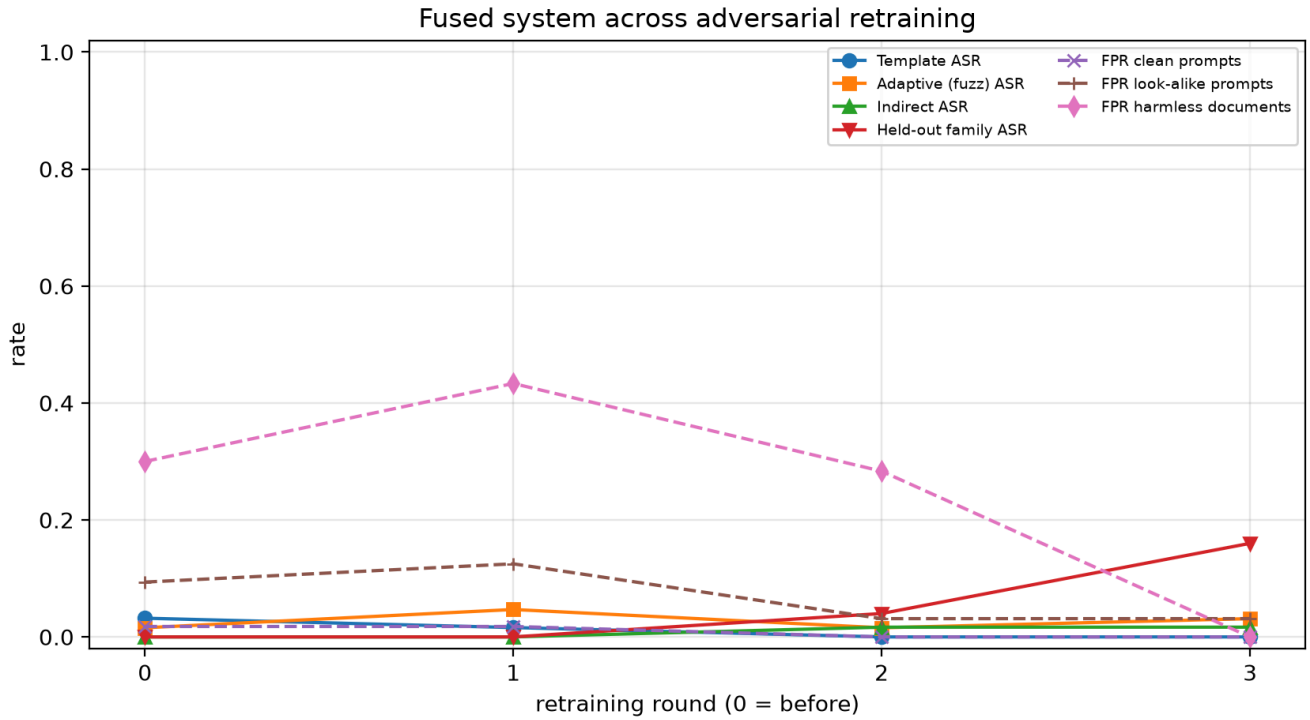


Figure 2. Attack success rate and false alarms across the three adversarial retraining rounds (seed 42).

An injection is hidden at 10%, 50% or 90% of a harmless document of 2,000, 6,000 or 12,000 characters. Cells show the share of injections detected at positions 10% / 50% / 90%.

System	2,000 chars	6,000 chars	12,000 chars	Harmless documents flagged (2,000 / 6,000 / 12,000)
Fused system	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00	0.00 / 0.01 / 0.00
ModernBERT, chunked	1.00 / 1.00 / 0.99	0.97 / 0.99 / 1.00	0.99 / 1.00 / 0.99	0.00 / 0.00 / 0.00
ModernBERT, full context	0.96 / 0.96 / 0.86	0.28 / 0.29 / 0.19	0.01 / 0.01 / 0.00	0.00 / 0.00 / 0.00
ModernBERT, first 512 tokens	0.96 / 0.96 / 0.86	0.93 / 0.00 / 0.00	0.96 / 0.00 / 0.00	0.00 / 0.00 / 0.00

System	2,000 chars	6,000 chars	12,000 chars	Harmless documents flagged (2,000 / 6,000 / 12,000)
DistilBERT, chunked	0.99 / 0.97 / 0.96	0.96 / 0.99 / 0.96	0.97 / 0.93 / 0.94	0.01 / 0.03 / 0.17
DistilBERT, first 512 tokens	0.93 / 0.96 / 0.76	0.88 / 0.00 / 0.00	0.88 / 0.00 / 0.00	0.00 / 0.00 / 0.00

This is the clearest result. Truncation misses every injection beyond the first 512 tokens. Reading a whole long document in one pass, although ModernBERT accepts 8,192 tokens, fails beyond a few thousand characters because training used short texts. Reading it in overlapping chunks and taking the highest score detects 93% to 100% of injections at every position. ModernBERT with chunking raises almost no false alarms on harmless long documents, whereas DistilBERT with chunking flags 17% of harmless 12,000-character documents.

5.5 Transfer to other public datasets

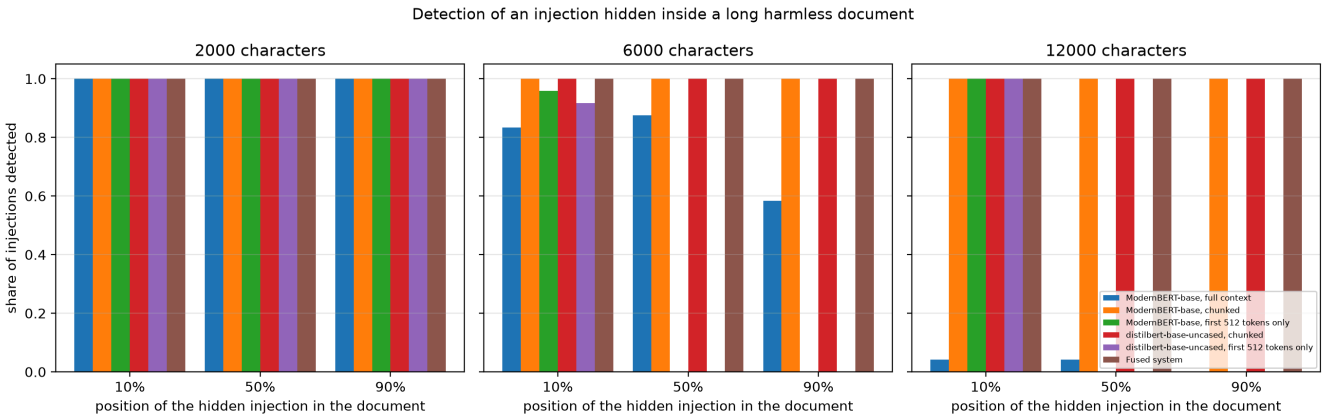


Figure 3. Long-document detection by document length and injection position (seed 42).

Dataset	Model	F1	ROC-AUC
xTRam1/safe-guard-prompt-injection	Pretrained guard model	0.912 ± 0.010	0.993 ± 0.004
xTRam1/safe-guard-prompt-injection	TF-IDF + LogReg	0.539 ± 0.011	0.702 ± 0.024

Dataset	Model	F1	ROC-AUC
xTRam1/safe-guard-prompt-injection	DeBERTa-v3-small	0.398 ± 0.067	0.497 ± 0.022
xTRam1/safe-guard-prompt-injection	DistilBERT	0.307 ± 0.004	0.430 ± 0.033
xTRam1/safe-guard-prompt-injection	ModernBERT alone (initial)	0.245 ± 0.030	0.362 ± 0.024
xTRam1/safe-guard-prompt-injection	Fused system, final	0.324 ± 0.021	0.480 ± 0.025
jackhhao/jailbreak-classification	Pretrained guard model	0.907	0.979
jackhhao/jailbreak-classification	DeBERTa-v3-small	0.721 ± 0.013	0.944 ± 0.025
jackhhao/jailbreak-classification	DistilBERT	0.730 ± 0.009	0.930 ± 0.036
jackhhao/jailbreak-classification	ModernBERT alone (initial)	0.752 ± 0.011	0.899 ± 0.063
jackhhao/jailbreak-classification	Fused system, final	0.728 ± 0.008	0.957 ± 0.027

The models trained here do not transfer. On xTRam1 their ROC-AUC is at or below 0.5 (chance), while the pretrained guard model, trained on much more data, scores 0.99. On the jailbreak dataset they rank well (0.90 to 0.96), but the threshold set on deepset flags most harmless prompts there. The limiting factor is the small training set, not the architecture.

5.6 Layer ablation (final fused system)

Variant	F1	ROC-AUC	Harmless documents flagged
All layers	0.940 ± 0.032	0.984 ± 0.003	0.117 ± 0.161
Without Layer 1	0.837 ± 0.058	0.964 ± 0.011	0.667 ± 0.262
Without Layer 2	0.937 ± 0.029	0.979 ± 0.000	0.100 ± 0.173
Without Layer 3	0.948 ± 0.009	0.985 ± 0.011	0.111 ± 0.164

Variant	F1	ROC-AUC	Harmless documents flagged
Layer 1 only	0.955 ± 0.006	0.976 ± 0.008	0.106 ± 0.169
Layer 2 only	0.743 ± 0.014	0.888 ± 0.002	0.683 ± 0.188
Layer 3 only	0.719 ± 0.001	0.865 ± 0.023	0.572 ± 0.096

Layer 1 carries the system. Layers 2 and 3 do not raise clean-test accuracy: removing Layer 3 even raises F1 slightly, and Layer 1 alone matches the full system on harmless documents. The fused score still gives calibrated probabilities (expected calibration error about 0.05) and the lowest adaptive attack success among the trained models.

5.7 Decision policy and speed

Input set	Allowed	Sent to review	Blocked
Clean test attacks	0.239 ± 0.092	0.000	0.761 ± 0.092
Clean test harmless prompts	1.000	0.000	0.000
Look-alike harmless prompts	0.990 ± 0.018	0.000	0.010 ± 0.018
Harmless documents	0.883 ± 0.161	0.000	0.117 ± 0.161
Template attacks (red team)	0.007 ± 0.011	0.000	0.993 ± 0.011
Attacks hidden in documents	0.067 ± 0.060	0.000	0.933 ± 0.060

The review band was empty in every final run because the 1%-false-positive threshold was not above the 95%-recall threshold, so the three-way policy reduced to allow or block.

Speed per input (mean over seeds): TF-IDF 1.0 ms, DistilBERT 5.5 ms, DeBERTa-v3-small 14.6 ms, ModernBERT 18.7 ms, Layer 2 12.1 ms, Layer 3 55.1 ms, fusion 1.5 ms, fused system in total 87.6 ± 8.8 ms. A 12,000-character document takes about 1.1 s with the fused system, 0.26 s with ModernBERT chunked and 0.10 s with DistilBERT chunked.

5.8 Hypotheses stated before the run

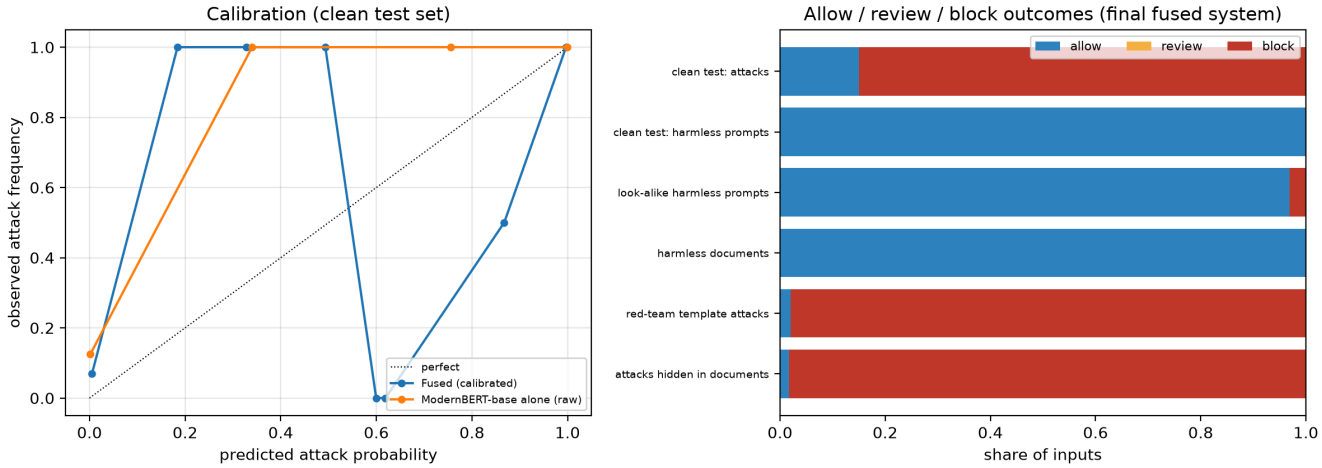


Figure 4. Calibration and allow/review/block policy of the fused system (seed 42).

1. Harmless documents flagged below 5%: not met on average (11.7% ± 16.1%, with a large spread between seeds), although it fell from 35.6% in round 0.
2. Full-context ModernBERT beats a 512-token model on 12,000-character documents: not met. Full context detected about 1% there; only chunked reading worked.
3. The adaptive attacker raises attack success on single models more than on the fused system: met for DistilBERT and DeBERTa (44% and 50% against 1.6%); ModernBERT alone is close to the fused system (4.7% against 1.6%).
4. The held-out homoglyph and foreign-language operators are the most successful: not met (1.4% and 0%).

6. Comparison with Other Models

No single model wins on every measure: DeBERTa has the highest clean F1, the pretrained guard model transfers best, and the fused system is hardest to bypass. Values are means over three seeds from Section 5.

Model	Clean F1	Adaptive attack success	Harmless 12,000-char documents flagged (chunked)	ROC-AUC on xTRam1 / jackhhao	ms per input	Main strength	Main weakness
TF-IDF + LinearSVM	0.834	0.958	not tested	0.680 / 0.906	1.0	Tiny, fast, easy to explain	Collapses under any rewording

Model	Clean F1	Adaptive attack success	Harmless 12,000-char documents flagged (chunked)	ROC-AUC on xTRam1 / jackhhao	ms per input	Main strength	Main weakness
DistilBERT	0.935	0.438	0.17	0.430 / 0.930	5.5	Fast and accurate on the clean test	Often bypassed; false alarms on long documents
DeBERTa-v3-small	0.945	0.495	not tested	0.497 / 0.944	14.6	Highest clean F1	Most bypassed; one run collapsed in version 1
Pretrained guard model	0.537	0.000	not tested	0.993 / 0.979	not measured	Best transfer, never bypassed	Flags 36% of harmless deepset prompts
ModernBERT alone	0.941	0.047	0.00	0.362 / 0.899	18.7	Long-context encoder, no long-document false alarms	Does not transfer to other datasets
Fused system (proposed)	0.940	0.016	0.00	0.480 / 0.957	87.6	Lowest adaptive attack success, calibrated score	Slowest, no accuracy gain over Layer 1

Takeaways:

- Classical models are cheap but unusable against an adaptive attacker (96% bypassed).
- Small transformers match the proposed system on the clean test, so a clean-test score alone cannot justify a more complex system.
- The proposed system earns its cost only through robustness to adaptive attacks and reliable long-document handling.
- A pretrained guard model trained on far more data is the better starting point for transfer, and combining it with the layers here is a natural next step.

7. Discussion: Pros, Cons, Limitations and Ethics

The layered design buys robustness at the price of complexity, latency and dependence on good attack data. No detector can guarantee safety, so the results are risk reduction, not prevention.

7.1 Pros and cons of the proposed system

Pros	Cons
Lowest adaptive attack success among the trained models (1.6%)	88 ms per short input and about 1.1 s for a 12,000-character document
Finds injections anywhere in long documents (100% in the tests)	Layers 2 and 3 do not improve clean-test accuracy
Calibrated probability (calibration error about 0.05)	Only 76% of clean-test attacks blocked at the chosen threshold
Collapse detection and retries make training failures visible	Review band empty; policy reduces to allow or block
Explains detections token by token	12% of harmless documents still flagged
Complementary layers: the statistical layer needs no attack labels	Does not transfer to other public datasets
Reproducible, three-seed evaluation with all tables saved	Needs a GPU for training and a 15 GB-class card at this size

7.2 Limitations

- Small training data: a few hundred real prompts plus generated text. This, not the architecture, explains the failed transfer to xTRam1.
- Small test set: 116 prompts (60 attacks); one prompt moves recall by about 1.7 points. Many differences between models are within noise.
- Generated benign data: harmless documents and attack payloads are written from templates. The four unseen document formats test generalisation only to a limited degree.
- Weak retraining signal: the attacker found 0 to 7 bypasses among 210 seeds per round, and retraining slightly raised held-out multilingual attack success.

- Threshold mismatch: thresholds tuned on a validation mix of easy generated attacks reach only 76% recall on real test attacks, and the three-way policy collapsed to two outcomes.
- Pretrained guard model: its strong transfer scores may reflect overlap with its own training data, so it is a reference point, not a perfectly fair opponent.
- Input-stage detection does not replace least-privilege design, output filtering or human approval for risky actions.

7.3 Ethical considerations

- Attack generation was done only in a private setup against the project's own pipeline.
- Attack payloads are harmless canary instructions, and no harmful content was generated or published.
- Datasets were used under their licences, and no private user data was collected.
- Blocking legitimate users has a cost, so false-positive rates are reported next to detection rates.

8. Future Work and Conclusion

8.1 Future work

- Train on much more real data, for example the public training splits of xTRam1 and jailbreak datasets, and evaluate on separate test splits, to fix the failed transfer.
- Combine the layers with a pretrained guard model as an additional feature.
- Tune the decision thresholds on harder validation attacks so that recall at the operating point matches the target and the review band is used.
- Cut cost: drop or distil the perplexity layer, which takes 55 of 88 milliseconds and adds no accuracy.
- Add an LLM-driven attacker such as PAIR [6] and test on agent benchmarks with real tool calls such as AgentDojo [8].
- Extend to more languages and to injections hidden in images.
- Study architectures that separate instructions from data instead of filtering text.

8.2 Conclusion

This project designed, built and evaluated a layered guardrail against prompt injection, and tested it more strictly than the first version: three seeds, an adaptive attacker, long documents, unseen formats and other datasets. Its measured strengths are long-document detection (chunked reading finds 93% to 100% of hidden injections with almost no false alarms, while truncation misses every injection beyond 512 tokens) and robustness to an adaptive attacker (1.6% success against the fused system, against 44% to 50% for DistilBERT and DeBERTa). Its measured limits are equally clear: clean-test accuracy is not better than a retrained DistilBERT, Layers 2 and 3 add cost without accuracy, 12% of harmless documents are still flagged, and the models do not transfer to another public dataset. The honest conclusion is that a modern encoder with chunked reading and adversarial retraining is a strong, auditable foundation, and that more diverse real training data is the most important next step.

References

1. F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques for Language Models," NeurIPS ML Safety Workshop, 2022. arXiv:2211.09527.
2. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," AISec 2023. arXiv:2302.12173.
3. Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Prompt Injection Attacks and Defenses in LLM-Integrated Applications," 2023 (later published as "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," USENIX Security 2024). arXiv:2310.12815.
4. S. Schulhoff et al., "Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs Through a Global Scale Prompt Hacking Competition," EMNLP 2023. arXiv:2311.16119.
5. A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," 2023. arXiv:2307.15043.
6. P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking Black Box Large Language Models in Twenty Queries," 2023. arXiv:2310.08419.
7. J. Yi et al., "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models," 2023. arXiv:2312.14197.

8. E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," NeurIPS 2024 Datasets and Benchmarks. arXiv:2406.13352.
9. G. Alon and M. Kamfonas, "Detecting Language Model Attacks with Perplexity," 2023. arXiv:2308.14132.
10. B. Warner et al., "Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference," 2024. arXiv:2412.13663.
11. P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing," ICLR 2023. arXiv:2111.09543.
12. V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2019. arXiv:1910.01108.
13. L. Wang et al., "Multilingual E5 Text Embeddings: A Technical Report," 2024. arXiv:2402.05672.
14. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP-IJCNLP 2019. arXiv:1908.10084.
15. Qwen Team, "Qwen2.5 Technical Report," 2024. arXiv:2412.15115.
16. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," KDD 2016. arXiv:1603.02754.
17. J. Platt, "Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods," in Advances in Large Margin Classifiers, MIT Press, 1999.
18. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS 2017. arXiv:1705.07874.

Verification note: the title and arXiv number (or publisher, for [17]) of all 18 references were checked by web search against arXiv, USENIX, NeurIPS or the author's page. Every entry is a peer-reviewed paper, an arXiv technical report from the model's authors, or a standard book chapter. Reference [3] appeared first as "Prompt Injection Attacks and Defenses in LLM-Integrated Applications" and was published at USENIX Security 2024 as "Formalizing and Benchmarking Prompt Injection Attacks and Defenses"; the arXiv number is the same.

Datasets and models: deepset/prompt-injections, xTRam1/safe-guard-prompt-injection, jackhhao/jailbreak-classification and protectai/deberta-v3-base-prompt-injection-v2 (Hugging Face Hub); model weights answerdotai/ModernBERT-base, intfloat/multilingual-

e5-small, Qwen/Qwen2.5-0.5B, microsoft/deberta-v3-small, distilbert-base-uncased.